Mid-term Project: Data Recovery on FAT32, exFAT, NTFS, Ext4

Data Safety and Recovery

1. Objectives

Each group recovers data from three logically damaged disk images on an assigned file system, by analyzing structures at the byte level and writing a program that performs the recovery operations. The **technical report** is graded as the theory part, and the **program** is graded as the practical part.

After the project, students will be able to:

1. Decode the relevant structures of the assigned file system with a hex editor.
2. Identify exactly which bytes are damaged and which sources of redundancy remain usable.
3. Write a recovery program that works correctly even on images it has never seen, including reassembling file fragments by content.
4. Write a report explaining the theoretical basis of every operation, with offset evidence.

2. Organization and schedule

- **Groups:** 3–4 students; each group is assigned one of the four file systems by lottery.
- **Images:** each group receives 3 images (one per scenario), generated by the lecturer's script with seed = group code. The lecturer also keeps a set of hidden images (same type of damage, different seed) to run each group's program during grading.
- **Program skeleton:** provided together with the assignment (Section 3).
- **Milestones:**
    a. Students have two weeks to complete the project
    b. Oral defense: 10–15 minutes per group.

3. General rules

Image parameters: 512 MB, cluster/block 4 KB, volume images (byte 0 is the boot sector).

File system	Creation command (Linux, reference)	Metadata checksums
FAT32	`mkfs.fat -F 32 -S 512 -s 8`	None
exFAT	`mkfs.exfat -c 4K`	Use the functions provided in the program skeleton
NTFS	`mkntfs -Q -c 4096 -s 512`	Fixup: handled by students
Ext4	`mkfs.ext4 -b 4096 -O ^metadata_csum`	Disabled

Reduced dataset: 5 files (2 JPEG, 1 PNG, 1 PDF, 1 text file under 600 B) in a subdirectory `DATA`; ASCII names. In scenario 3, the larger JPEG file is split into 2 fragments. The `DATA` directory is kept small enough to fit in a single unit:

- FAT32, exFAT: exactly 1 cluster.

- NTFS: $INDEX_ROOT only, no INDX block (the root directory still uses one INDX block).
- Ext4: 1 directory block, all extents at depth 0.

Program skeleton provided by the lecturer (Python): command-line interface, functions to read bytes at an offset, hex dump, SHA-256 computation, logging, exFAT checksum functions, and a self-test script on sample images. The skeleton contains **no** file-system decoding code.

Tools

- **Allowed:** a hex editor; the Python standard library; the program skeleton; reference-purpose tools (dumpe2fs, ntfsinfo, `fsck.fat -n`, dump.exfat).
- **Forbidden in the program:** filesystem parsing libraries (pytsk3, dfvfs, pyfatfs, libntfs, libext2fs…) and calling external tools through system commands.
- **Forbidden during recovery:** chkdsk /f, fsck in repair mode, TestDisk, PhotoRec, R-Studio (allowed only for verification, and their use must be stated in the report).

Program requirements

1. `recover --image <image> --info` prints the volume parameters and the directory tree; `recover --image <image> --case {1,2,3} --out <directory>` performs recovery. The bonus part also uses `--repair <new image>` or `--corrupt <spec>` / `--case ext`.
2. Do not hard-code offsets or values specific to the group's image, because the program will be run on hidden images.
3. Never write to the input image; the log prints every step with the offset and the value read.
4. Process each image within at most 10 minutes.

4. Scenario 1 — Wrong boot sector parameters

Three key parameter fields are overwritten with values that are wrong but formally valid, in **both the primary copy and every backup**; all other structures are intact.

File system	Fields made wrong	Backup also wrong	Note
FAT32	BPB_SecPerClus (0x0D), BPB_RsvdSecCnt (0x0E), BPB_FATSz32 (0x24)	Sector 6	None
exFAT	FatOffset (0x50), ClusterHeapOffset (0x58), SectorsPerClusterShift (0x6D)	Backup Boot Region (sectors 12–23)	Recompute the Boot Checksum
NTFS	Sectors per cluster (0x0D), $MFT LCN (0x30), MFT record size (0x40)	Last sector of the volume	None
Ext4	s_log_block_size (0x18), s_blocks_per_group (0x20), s_inodes_per_group (0x28)	All backup superblocks	None

Requirements: state the correct value of each field with reasoning based on offsets; the program must derive these values by itself and extract every file.

5. Scenario 2 — Wrong directory entries

In the `DATA` directory, three target files (different formats and sizes, all contiguous) lose their **name → data** link.

File system	Made wrong	Still intact
FAT32	First cluster (high 0x14, low 0x1A) in the short entry	Name, size, FAT chain
exFAT	FirstCluster (0x14) in the Stream Extension; SetChecksum recomputed to match	Name, DataLength, Allocation Bitmap
NTFS	MFT reference (6 bytes) in the index entry; name characters in the record's $FILE_NAME zeroed (length kept)	Index entry key (name, size, timestamps), MFT record and runlist
Ext4	Inode number (0x0) in the directory entry	Name, file_type, inode and extents

Requirements: find the "orphan" candidates (allocated but referenced by nothing), match each name with its data correctly and state the matching criteria; extract the three files.

6. Scenario 3 — Wrong location chain

A JPEG file is fragmented into exactly 2 pieces, the second piece lying within [64] clusters after the first. Only the start location and size remain; the information about the second piece and its allocation bits is erased.

File system	Made wrong	Still intact
FAT32	The file's FAT chain zeroed in both tables	First cluster and size in the directory entry
exFAT	FAT chain zeroed, NoFatChain flag cleared, Allocation Bitmap bits of every cluster after the first cluster cleared	FirstCluster and DataLength
NTFS	Runlist reduced to the first run with a length of 1 cluster; $Bitmap bits of the rest cleared	Start LCN, real size of $DATA
Ext4	Only the first extent remains, with a length of 1 block; block bitmap bits of the rest cleared	ee_start of the first extent, i_size

The images are built outside the kernel, so the journal holds no old metadata copies of the target file.

Requirements: compute the number of missing clusters/blocks from the size; locate the second piece by content (JPEG structure, the FF D9 marker, successful decoding); list the final location chain and extract the file.

7. Bonus part (choose one)

Damage it yourself, recover it yourself: choose one scenario below, submit a short JSON specification, write `--corrupt <spec>` (changing only the declared metadata region) and `--case ext` to recover it.

Scenario	FAT32	exFAT	NTFS	Ext4
Wrong root directory pointer	BPB_RootClus (0x2C)	FirstClusterOfRootDirectory (0x60)	Header of MFT record 5 damaged	Inode 2 zeroed
Wrong file size	File size (0x1C)	DataLength, ValidDataLength	Real and initialized size of $DATA	i_size
Lost MBR partition table entry (whole-disk image)	Entry 0x1BE zeroed	Entry 0x1BE zeroed	Entry 0x1BE zeroed	Entry 0x1BE zeroed

The lecturer runs `--corrupt` on their own clean image and runs `--case ext` on a hidden image of the same scenario. Images are not exchanged between groups.

8. Deliverables

1. **PDF report** (at most 20 pages): a structure chapter covering only the fields actually used; for each scenario, the diagnosis, theoretical basis, pseudocode and hex evidence.
2. **Source code** built on the program skeleton, with a `README` stating who wrote which part.
3. **A `recovered/` directory** created by the program, with `SHA256SUMS`, plus the run logs on the group's images.
4. **Bonus part:** the JSON specification and the self-damaged image.

Oral defense: each member explains the part they wrote and reads, with a hex editor, a structure chosen by the lecturer.

9. Grading

Two independent components, each out of 100. Weight in the course grade: [__].

Theory (report)	Points	Practice (program)	Points
Structure chapter	25	`--info` on an intact image	15
Scenario 1	20	Scenario 1	25
Scenario 2	25	Scenario 2	25
Scenario 3	20	Scenario 3	25
Presentation, hex evidence	10	Code quality (no hard-coded offsets, logging, README)	10

- Each scenario in the theory component: diagnosis 40%, theoretical basis 40%, algorithm and limitations 20%.

- Each scenario in the practice component: group image 40%, hidden image 50%, correct names and paths 10%. A file is correct when its SHA-256 matches the original.
- **Bonus (added to practice):** up to 10 (4 for `--corrupt` matching the specification, 6 for recovery on the hidden image).
- **Penalties:** hard-coded offsets → 0 for the hidden-image part of that scenario; forbidden library/tool → 0 for that scenario in both components; report contradicting the log → theory points deducted for that scenario.